

Guía de Comandos Esenciales de MIPS Assembly para el Laboratorio

Este documento es una guía de referencia rápida con los comandos y conceptos de MIPS Assembly necesarios para resolver los ejercicios del laboratorio.

1. Estructura Básica de un Programa

Un programa en MIPS se divide en dos secciones principales:

- **.data**: Aquí se declaran los datos estáticos que el programa utilizará, como mensajes de texto o variables.
- **.text**: Aquí se escriben las instrucciones que el procesador ejecutará. La ejecución siempre comienza en la etiqueta **main**.

```
# Inicio de la sección de datos
.data
    # Aquí van las declaraciones de datos

# Inicio de la sección de código
.text
.globl main
main:
    # Aquí van las instrucciones del programa

    # No olvidar finalizar el programa
    li $v0, 10
    syscall
```

2. Declaración de Datos (en la sección **.data**)

La principal necesidad para el laboratorio es declarar los mensajes que se mostrarán al usuario.

- **.ascii** **"Tu texto aquí"**: Declara una cadena de caracteres (string) terminada con un carácter nulo.

```
.data
mensaje_bienvenida: .ascii "Bienvenido al programa de cálculo.\n"
prompt_numero: .ascii "Por favor, introduce un número: "
```

3. Llamadas al Sistema (**syscall**)

Para que nuestro programa interactúe con el exterior (leer del teclado, imprimir en pantalla), necesitamos pedirle servicios al sistema operativo a través de la instrucción `syscall`. El proceso siempre es el mismo:

- 1. Cargar el **código del servicio** en el registro `$v0`.
- 2. Cargar los **argumentos** necesarios en los registros `$a0`, `$a1`, etc.
- 3. Ejecutar `syscall`.

syscalls Esenciales:

Servicio	Código en <code>\$v0</code>	Argumentos	Resultado
Imprimir Entero	1	<code>\$a0</code> = El número a imprimir	
Imprimir String	4	<code>\$a0</code> = La dirección de memoria del string	
Leer Entero	5	(Ninguno)	El número leído se guarda en <code>\$v0</code>
Terminar Programa	10	(Ninguno)	

Ejemplo Práctico:

```
.data
    prompt: .asciiz "Introduce tu edad: "
    respuesta: .asciiz "\nTu edad es: "

.text
.globl main
main:
    # 1. Imprimir el string de solicitud
    li $v0, 4          # Código para imprimir string
    la $a0, prompt      # Argumento: la dirección de 'prompt'
    syscall

    # 2. Leer el número entero
    li $v0, 5          # Código para leer entero
    syscall             # El número introducido se guarda en $v0

    # 3. Guardar el número leído para usarlo después
    move $t0, $v0      # Movemos el resultado de $v0 a un registro temporal $t0

    # 4. Imprimir el string de respuesta
    li $v0, 4
    la $a0, respuesta
    syscall

    # 5. Imprimir el número que guardamos
    li $v0, 1          # Código para imprimir entero
    move $a0, $t0      # Argumento: el número que estaba en $t0
```

```
syscall

# 6. Finalizar
li $v0, 10
syscall
```

4. Instrucciones de Registros

- **li \$reg, valor** (Load Immediate): Carga una constante numérica directamente en un registro.

```
li $t0, 100      # $t0 = 100
```

- **la \$reg, etiqueta** (Load Address): Carga la dirección de memoria de una etiqueta (como un mensaje) en un registro.

```
la $a0, mensaje_bienvenida
```

- **move \$reg_destino, \$reg_fuente**: Copia el valor de un registro a otro.

```
move $t1, $v0     # Copia el valor de $v0 en $t1
```

5. Instrucciones Aritméticas

- **add \$rd, \$rs, \$rt** (Add): Suma el contenido de dos registros y guarda el resultado en un tercero.
 $\$rd = \$rs + \$rt$.

```
# Ejemplo: Sumar el contenido de $t0 y $t1, y guardarlo en $t2
add $t2, $t0, $t1
```

- **addi \$rt, \$rs, inmediato** (Add Immediate): Suma el contenido de un registro con un valor constante. $\$rt = \$rs + \text{valor}$.

```
# Ejemplo: Incrementar un contador en $t0
addi $t0, $t0, 1 # $t0 = $t0 + 1
```

6. Instrucciones de Control de Flujo (Saltos)

Estas instrucciones son la clave para crear bucles y lógica condicional (if/else). Funcionan con **etiquetas**.

- **etiqueta::** Un marcador en el código al que podemos saltar.
- **j etiqueta** (Jump): Salta incondicionalmente a la etiqueta especificada.

```
bucle:
    # ... cuerpo del bucle ...
    j bucle # Vuelve a empezar el bucle
```

- **beq \$rs, \$rt, etiqueta** (Branch on Equal): Salta a la etiqueta si los valores en los dos registros son iguales.

```
# Salir de un bucle si el contador $t0 llega a 10
beq $t0, 10, fin_del_bucle
```

- **bne \$rs, \$rt, etiqueta** (Branch on Not Equal): Salta si los valores son diferentes.
- **bgt \$rs, \$rt, etiqueta** (Branch on Greater Than): Salta si $\$rs > \rt . **(Esencial para el problema del número mayor).**

```
# Compara un nuevo_numero ($t1) con el mayor_actual ($t0)
# Si nuevo_numero > mayor_actual, actualizamos.
bgt $t1, $t0, es_mayor
# ... si no es mayor, no hace nada ...
es_mayor:
    move $t0, $t1 # Actualiza: mayor_actual = nuevo_numero
```

- **blt \$rs, \$rt, etiqueta** (Branch on Less Than): Salta si $\$rs < \rt . **(Esencial para el problema del número menor).**